

- AXI4-Lite 기반 SPI, I2C Peripheral 설계
- UVM 검증

+ Demo Video





: Contents

Chapter 1.

Introduction

- AXI



Chapter 2.

AXI4-Lite 기반 SPI Peripheral 설계

- AXI4-Lite & SPI
- Demo Video



Chapter 3.

AXI4-lite 기반 I2C Peripheral 설계

- AXI4-Lite & I2C
- Demo Video



Chapter 4.

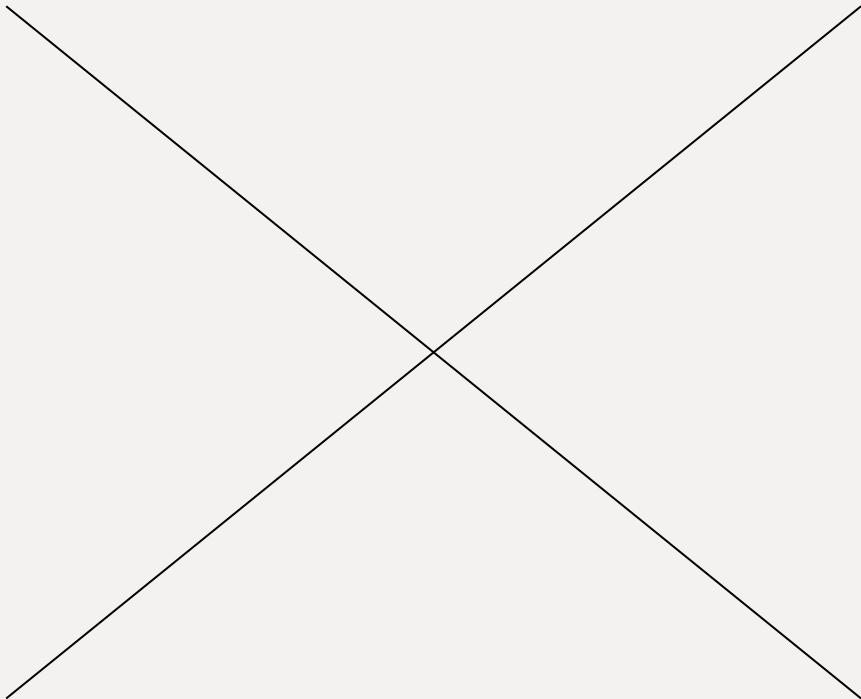
UVM

- SPI



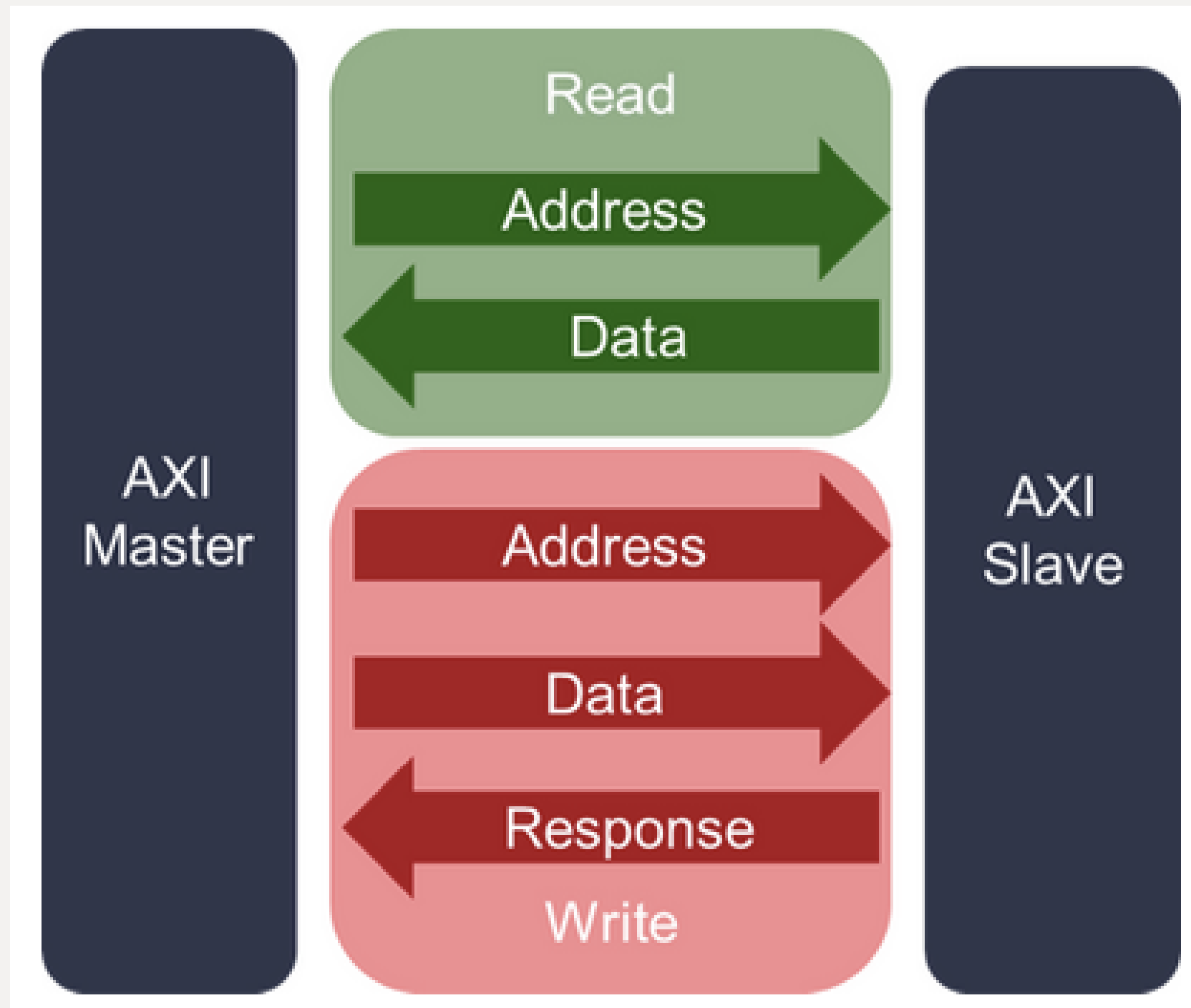
Chapter 5.

Trouble Shooting



Introduction

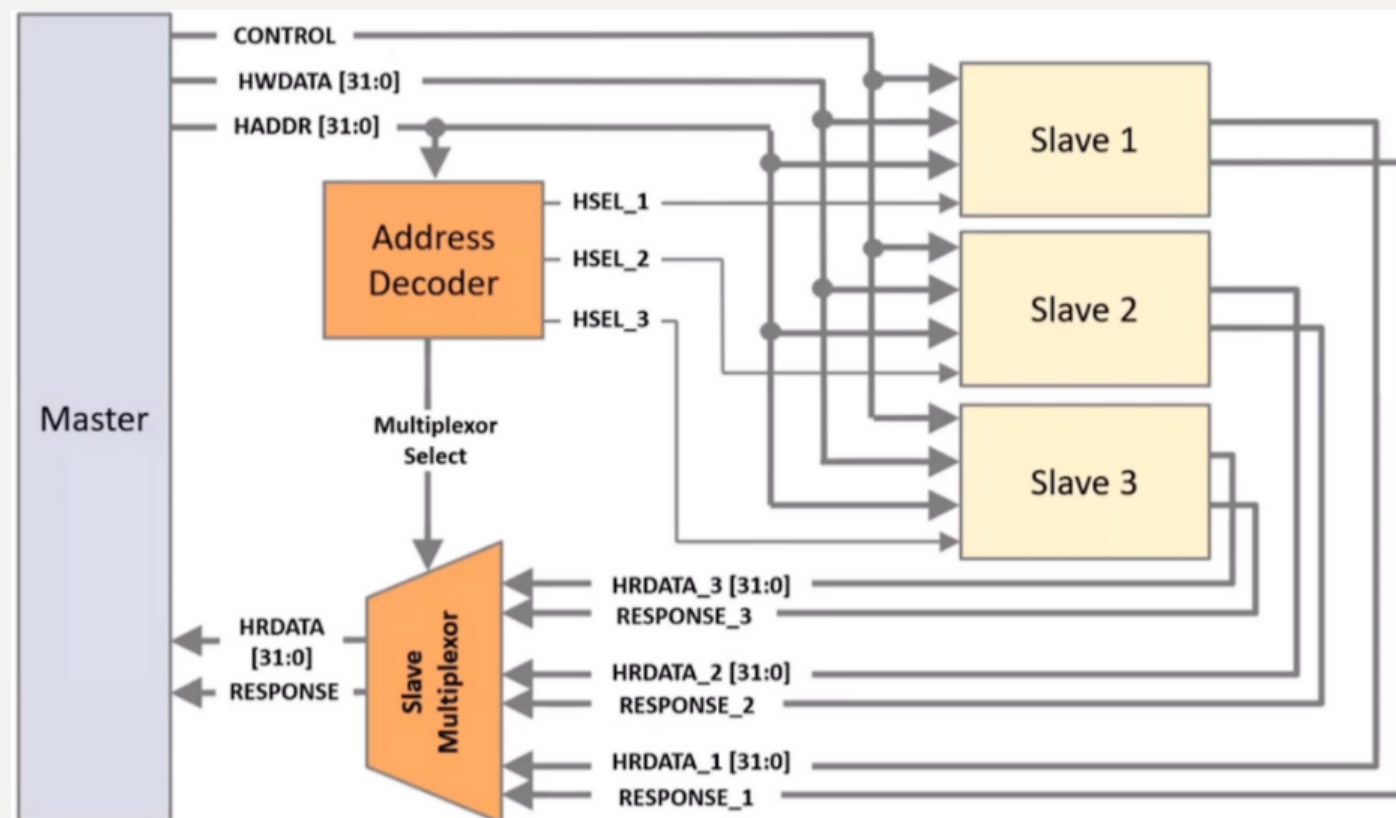
: Introduction



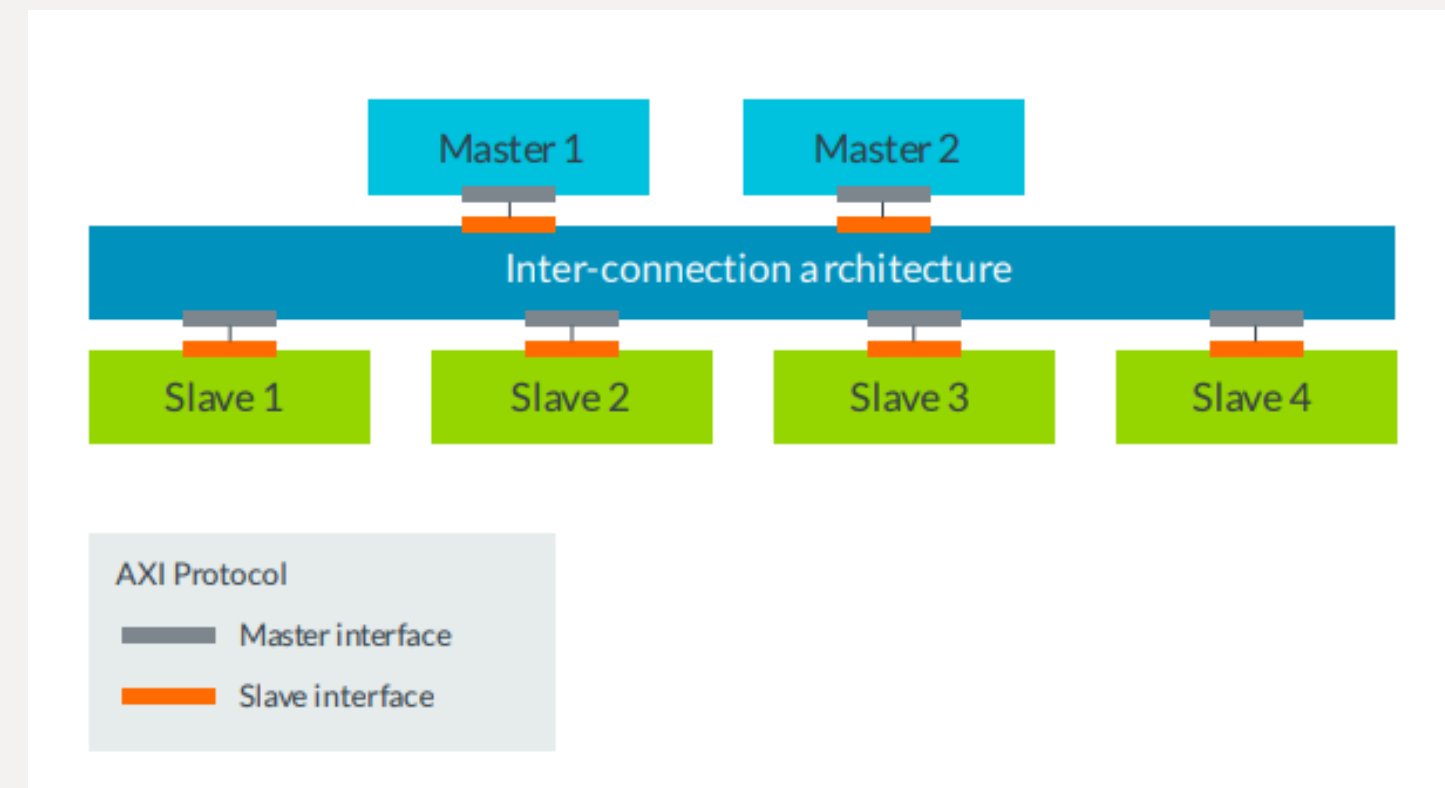
- *Protocol* *AXI - Advanced eXtensible Interface*
- *Character* Channel-based 통신
Valid - Ready handshake
Point-to-point direct 통신(bus x)
- *Channel*
 - Write Address(AW)
 - Write Data(W)
 - Write Response(B)
 - Read Address(AR)
 - Read Data(R)



: Instruction

AHB(APB)

APB와 같은 BUS는 모든 slave에
address, data를 뿌려주는 **broadcasting 방식**

AXI

master와 slave가 통신을 하고 싶으면
해당 slave 하나에만 보냄 ⇒ **point-to-point**
그 역할은 **interconnector**가 맡음
(Master는 interconnector를 통해 slave와 통신)

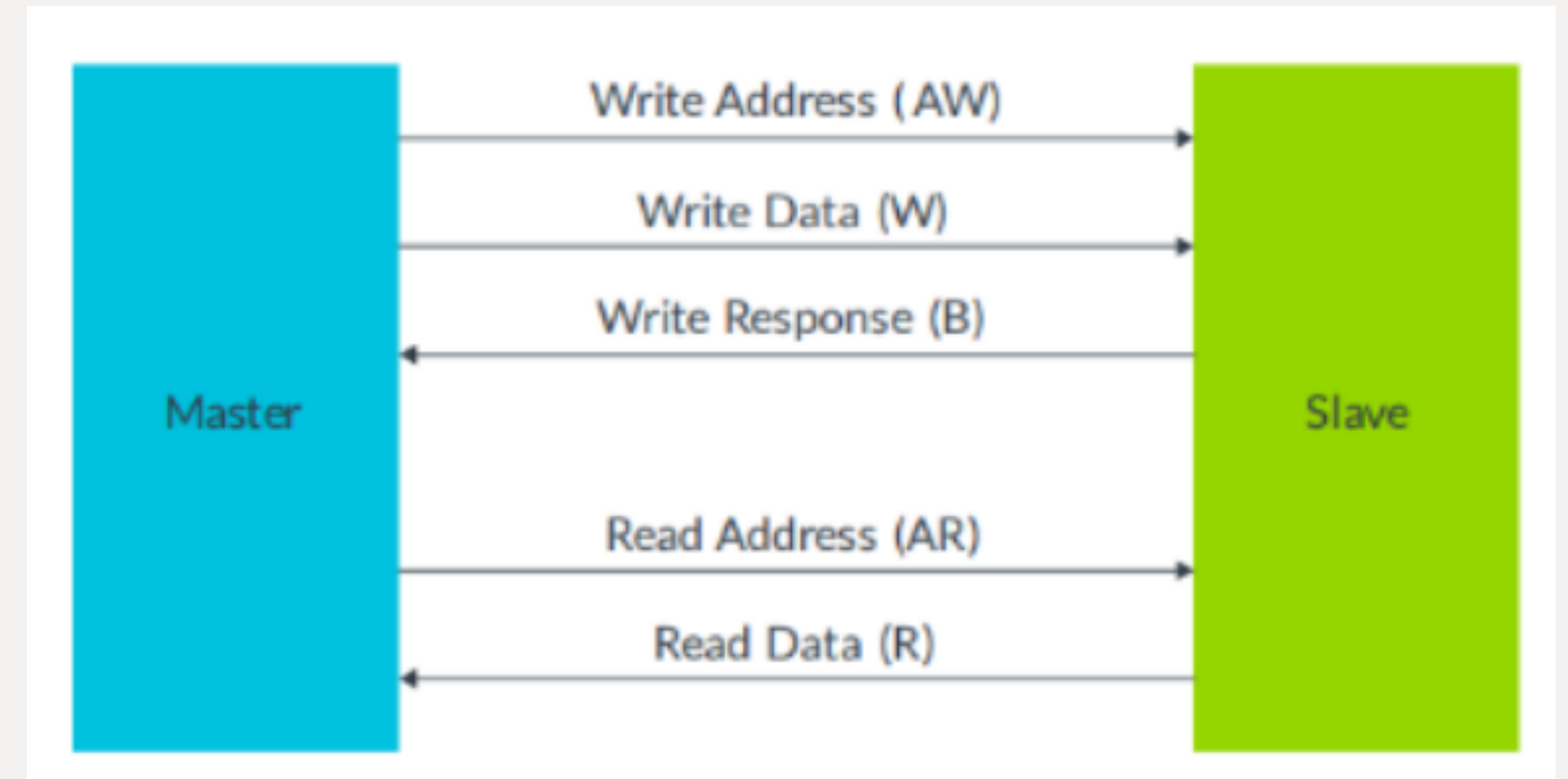
: Introduction



AXI protocol



Valid - Ready handshake

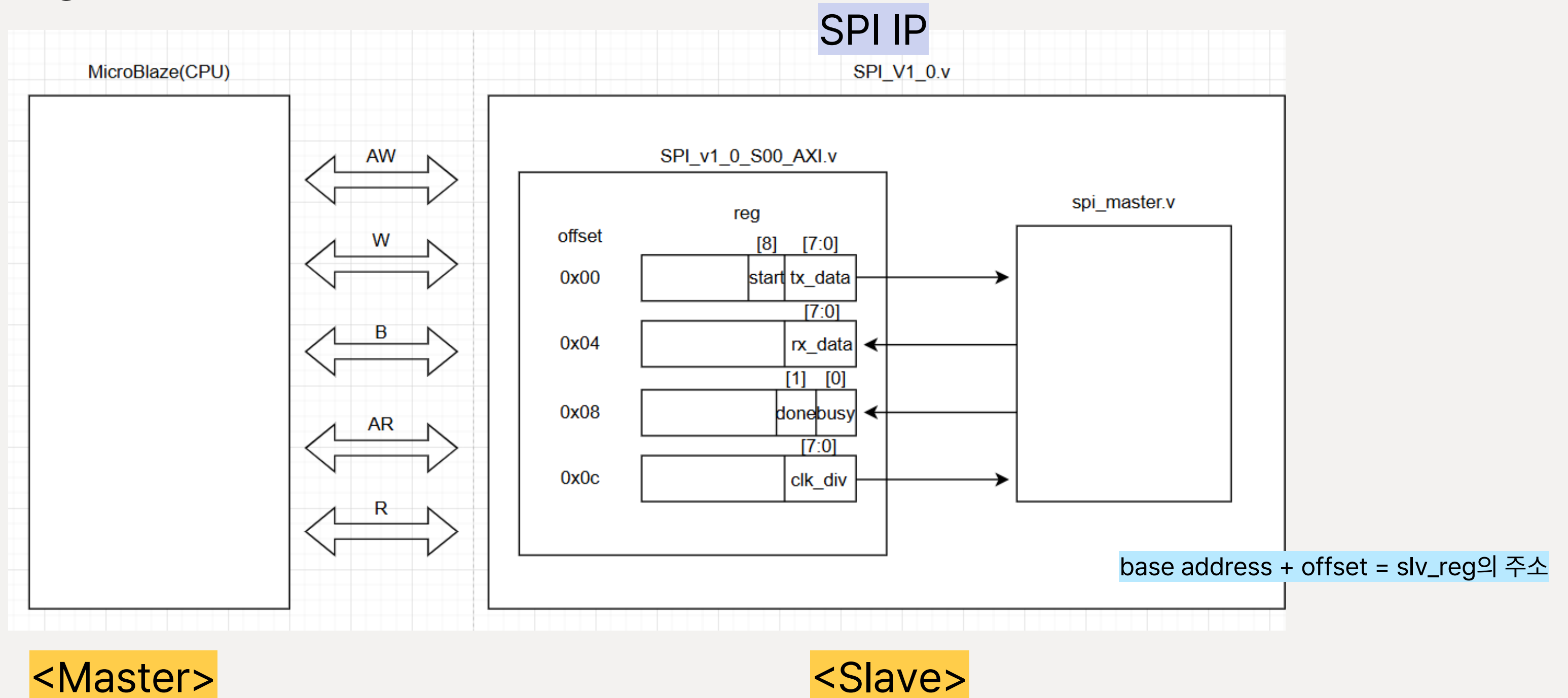


5 channels

AXI4-lite 기반 SPI Peripheral 설계

: AXI4-lite 기반 SPI Peripheral 설계

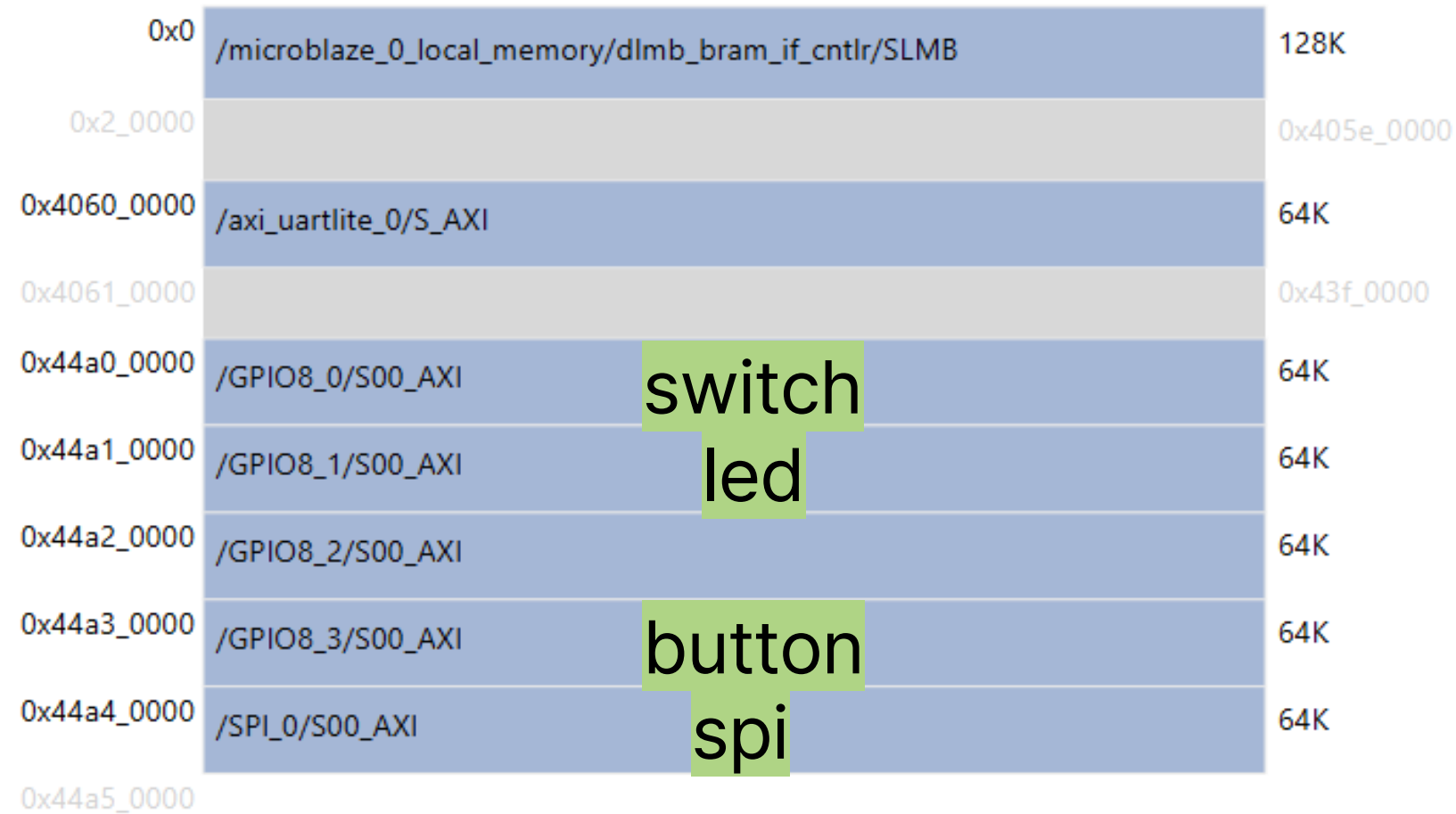
Block Diagram



: AXI4-lite 기반 SPI Peripheral 설계



Software



Address map

Application

SPI_FPGA

Driver

Button
driverLED
driver

common

HAL

GPIO

SPI

HW

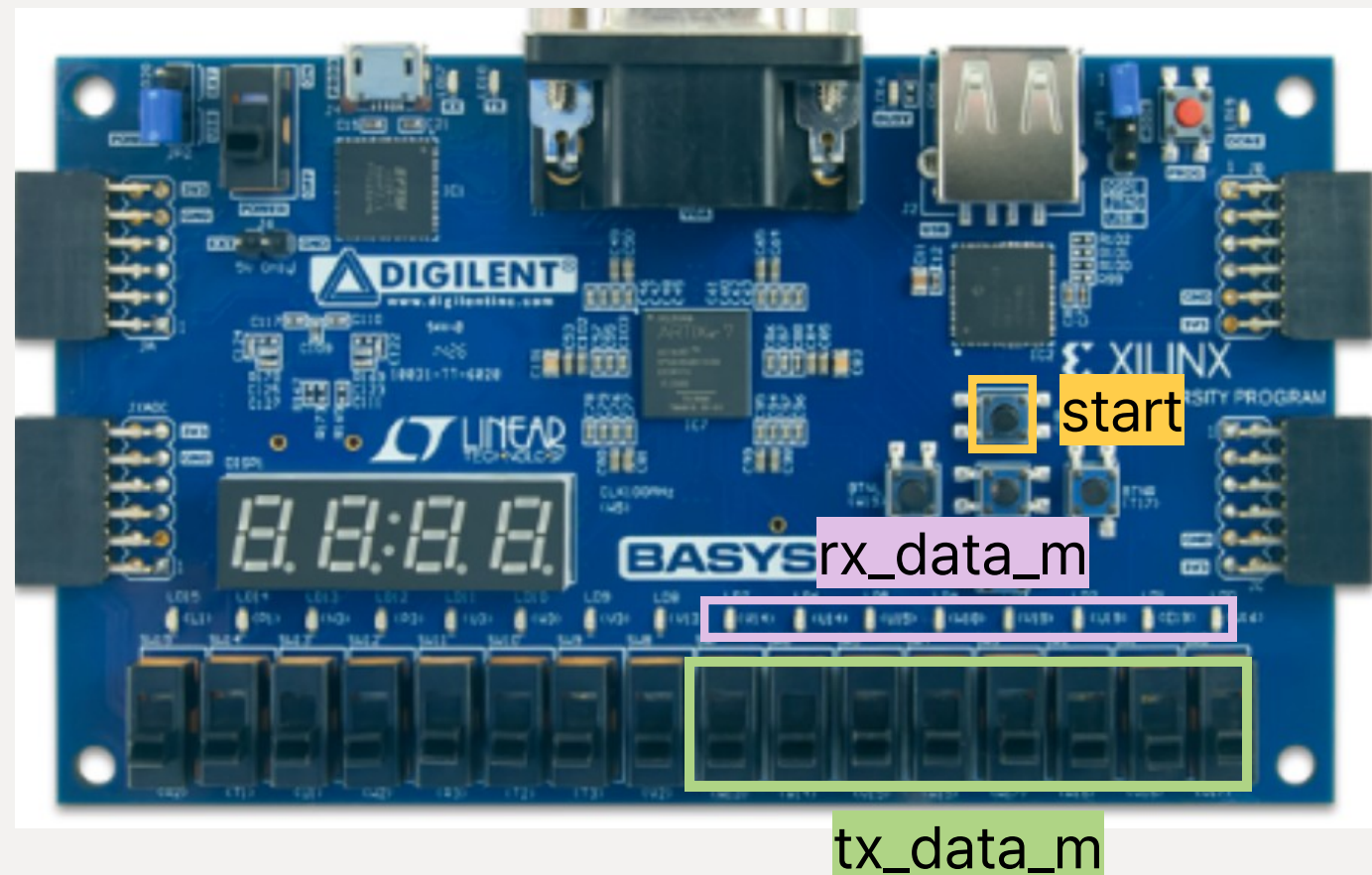
Button

LED

계층형 아키텍처

: AXI4-lite 기반 SPI Peripheral 설계

Software

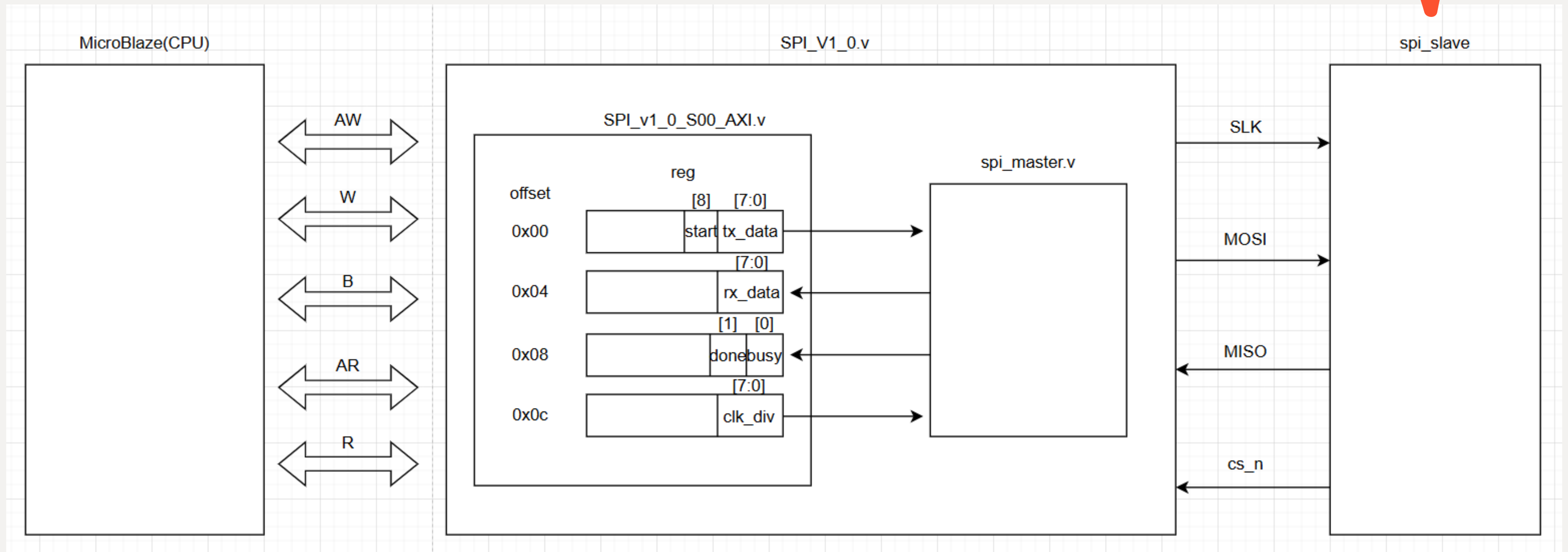


< Master >

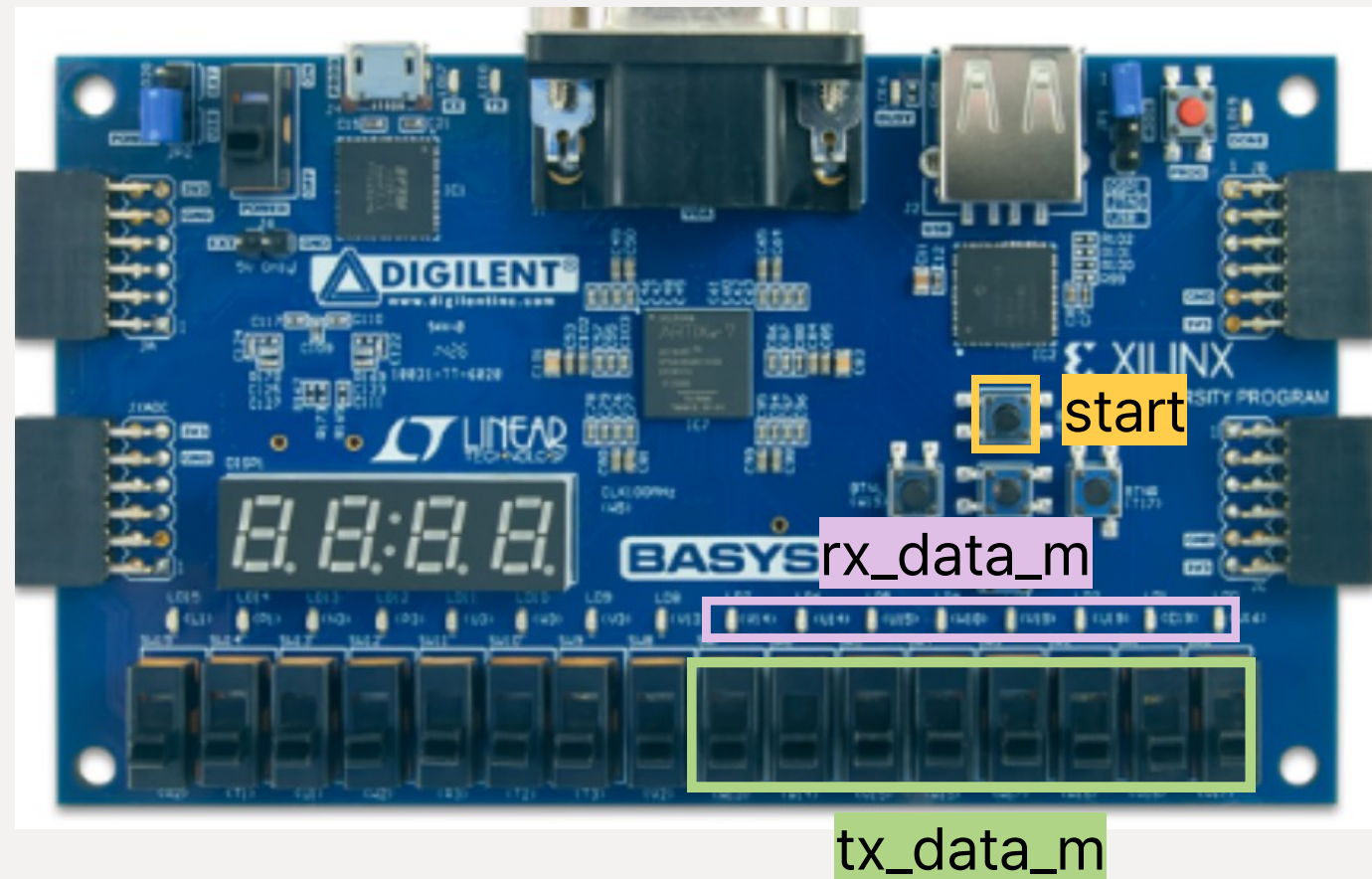
● Application SPI_FPGA

- 동작
- 버튼 push
 - sw(GPIOA)에 있는 data를 읽음
 - sw_data를 slave(FPGA) 장치로 전송
 - 동시에 슬레이브로부터 넘어오는 received_data를 받음
 - 받은 data를 led에 출력

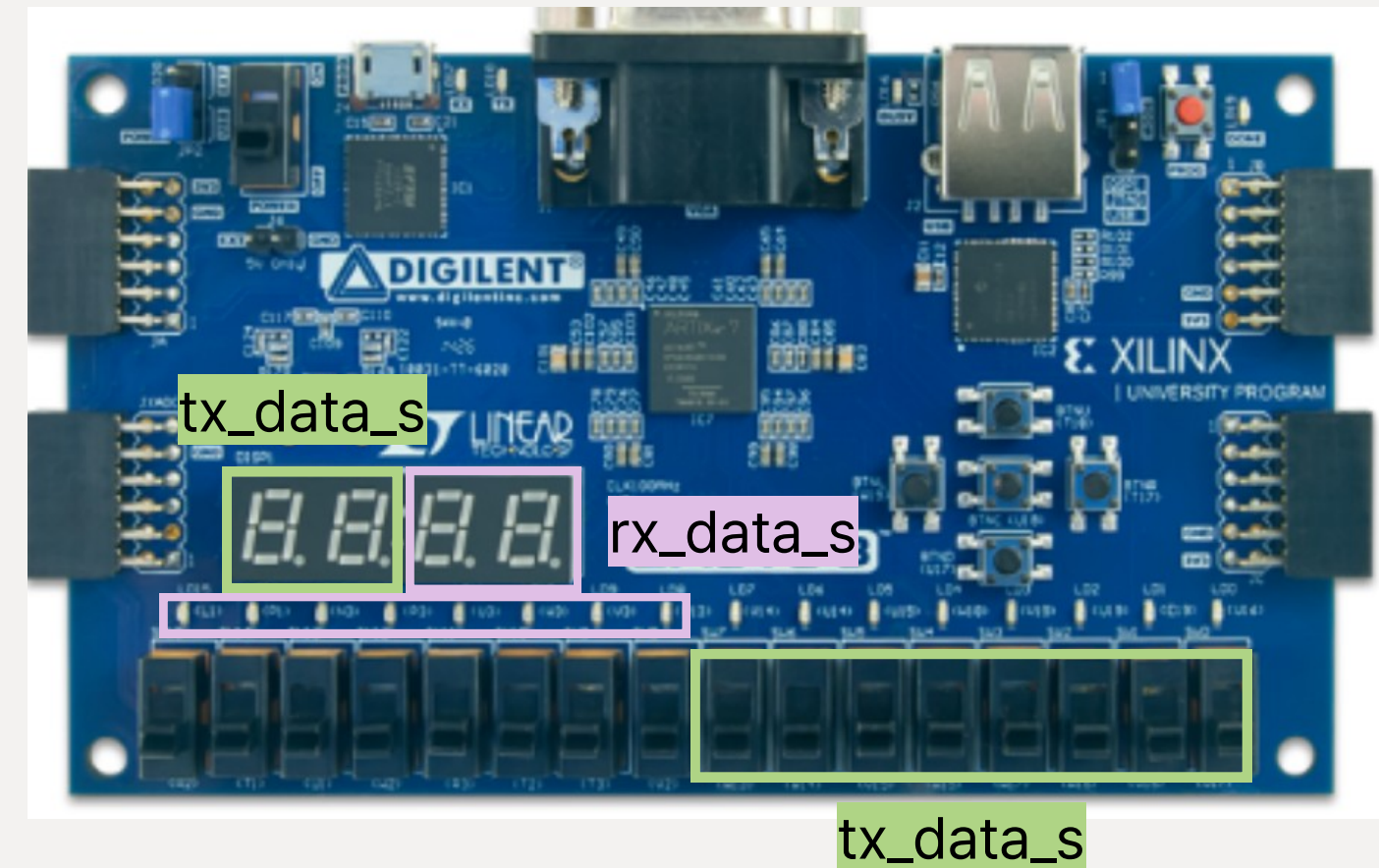
: AXI4-lite 기반 SPI Peripheral 설계



: AXI4-lite 기반 SPI Peripheral 설계



< Master >



< Slave >

Chapter 2

Demo

: AXI4-lite 기반 SPI Peripheral 설계

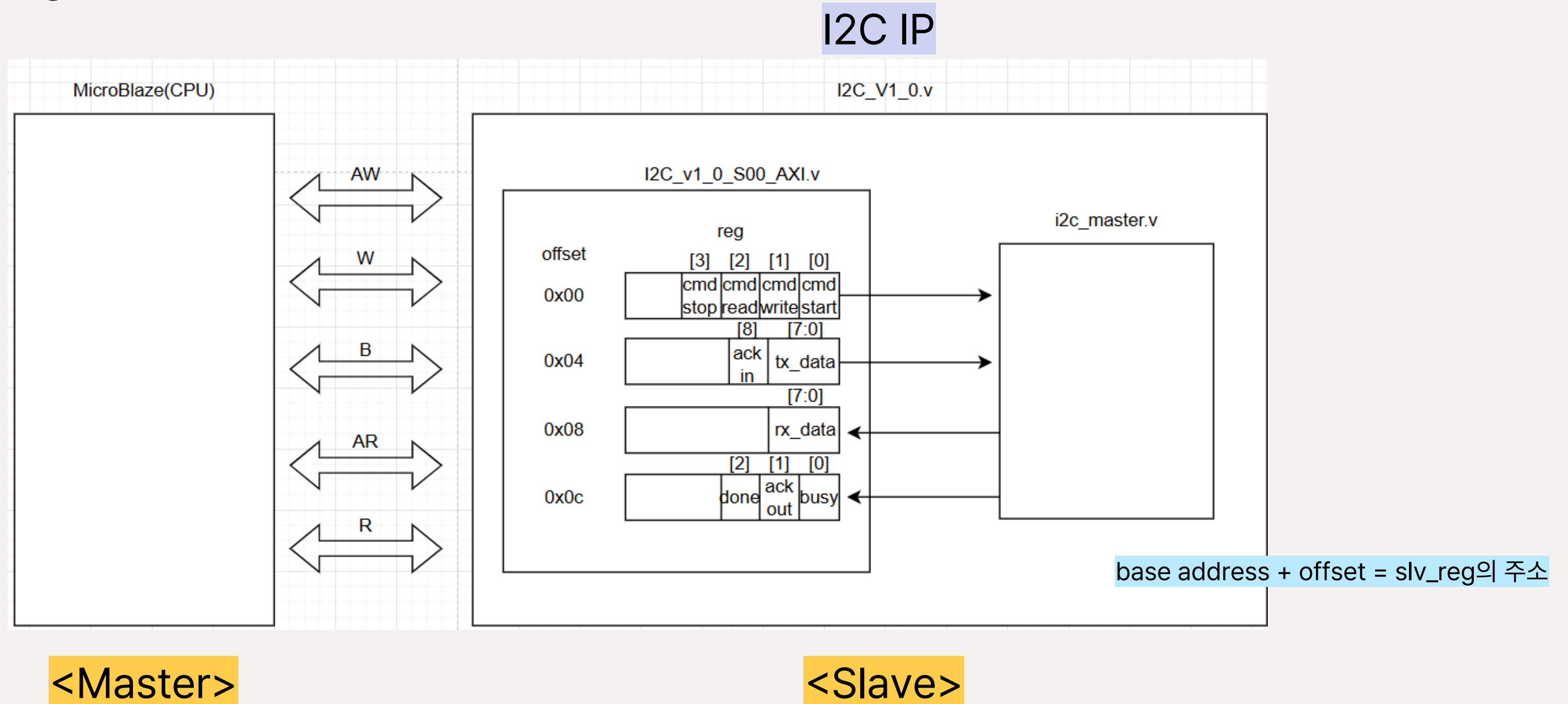


AXI4-lite 기반 I2C Peripheral 설계

: AXI4-lite 기반 I2C Peripheral 설계

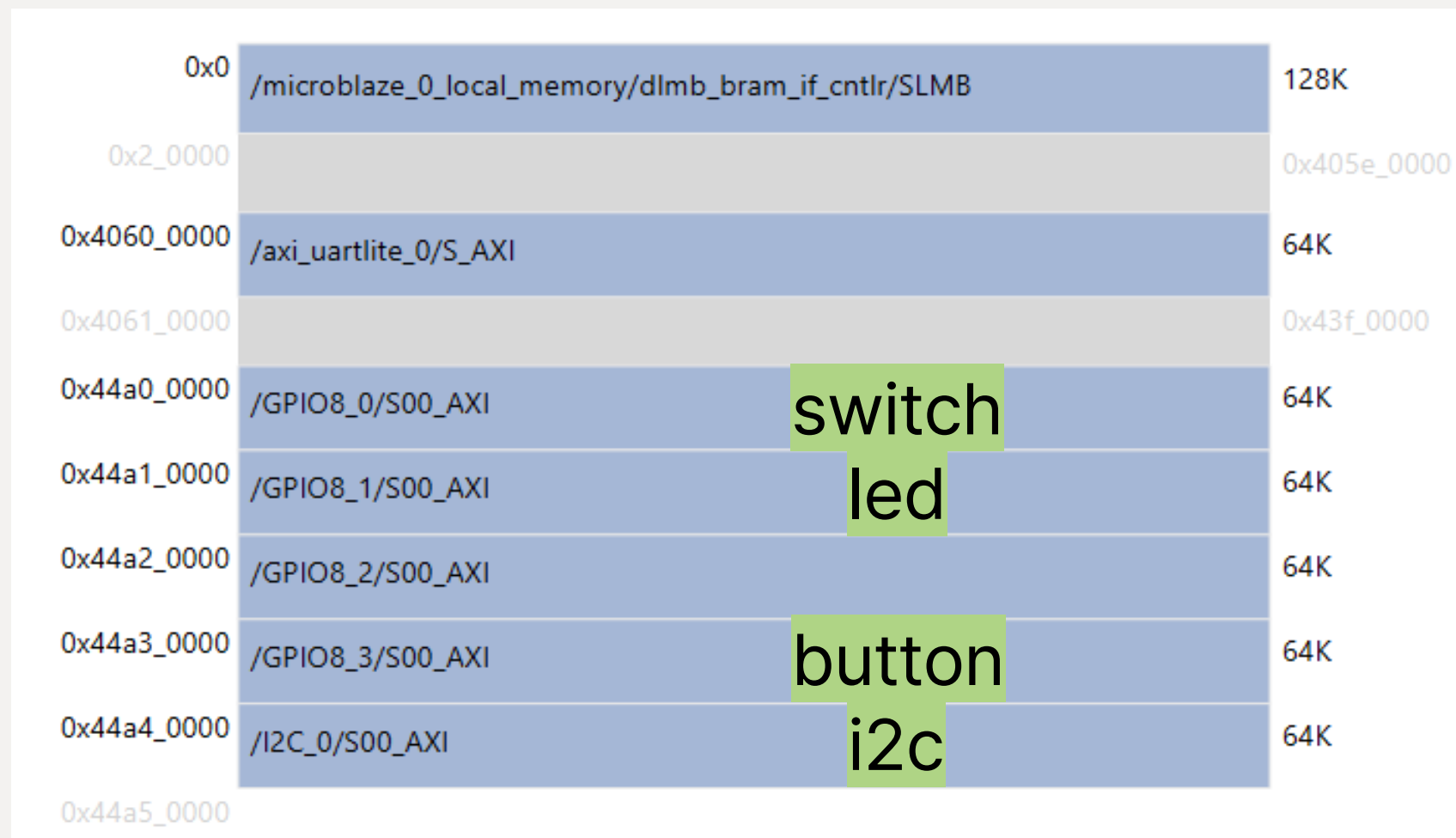


Block Diagram



: AXI4-lite 기반 I2C Peripheral 설계

Software



Address map

Application

I2C_FPGA

Driver

Button
driver

LED
driver

common

HAL

GPIO

I2C

HW

Button

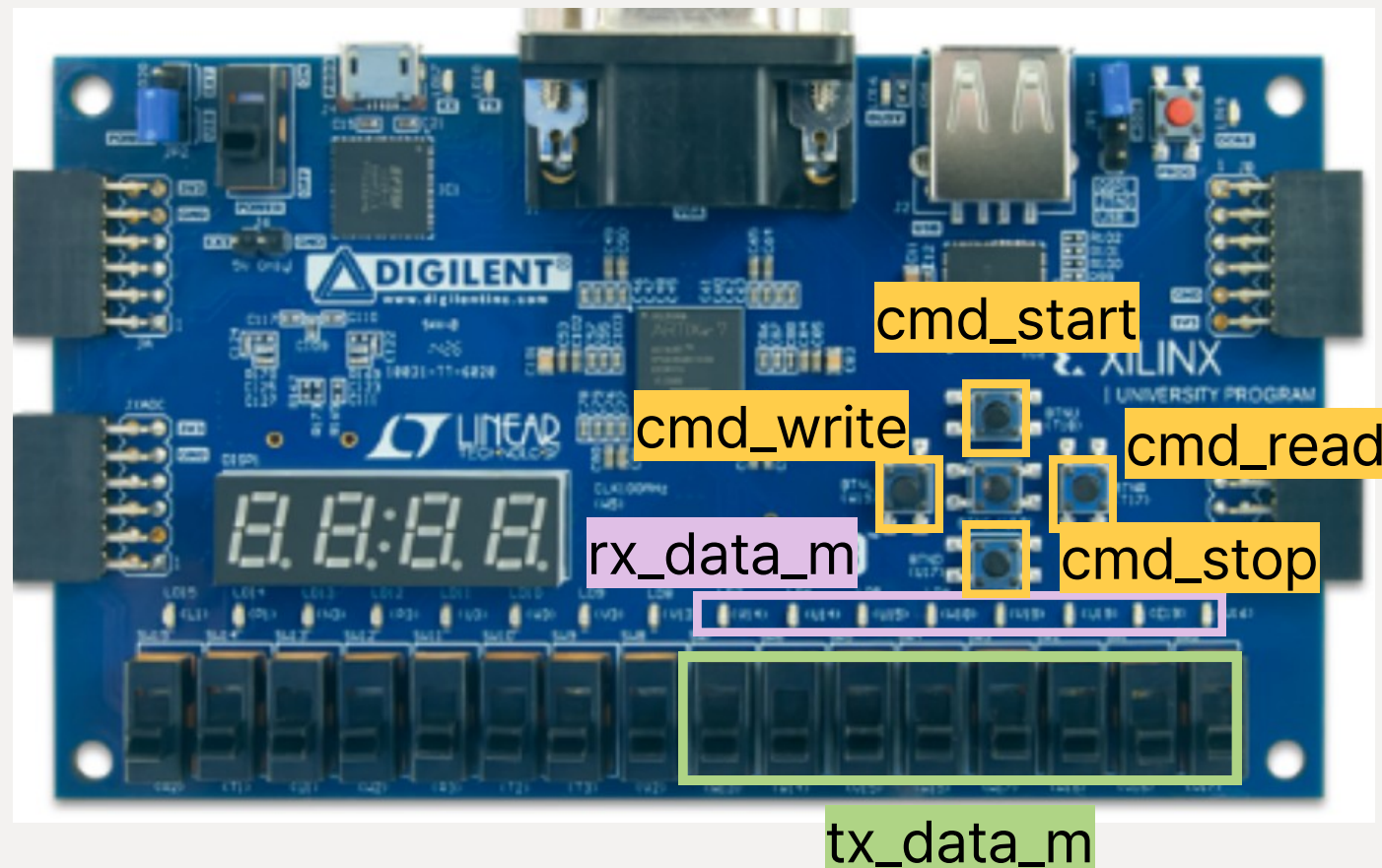
LED

계층형 아키텍처

: AXI4-lite 기반 I2C Peripheral 설계



Software



< Master >

- Application I2C_FPGA

- 동작

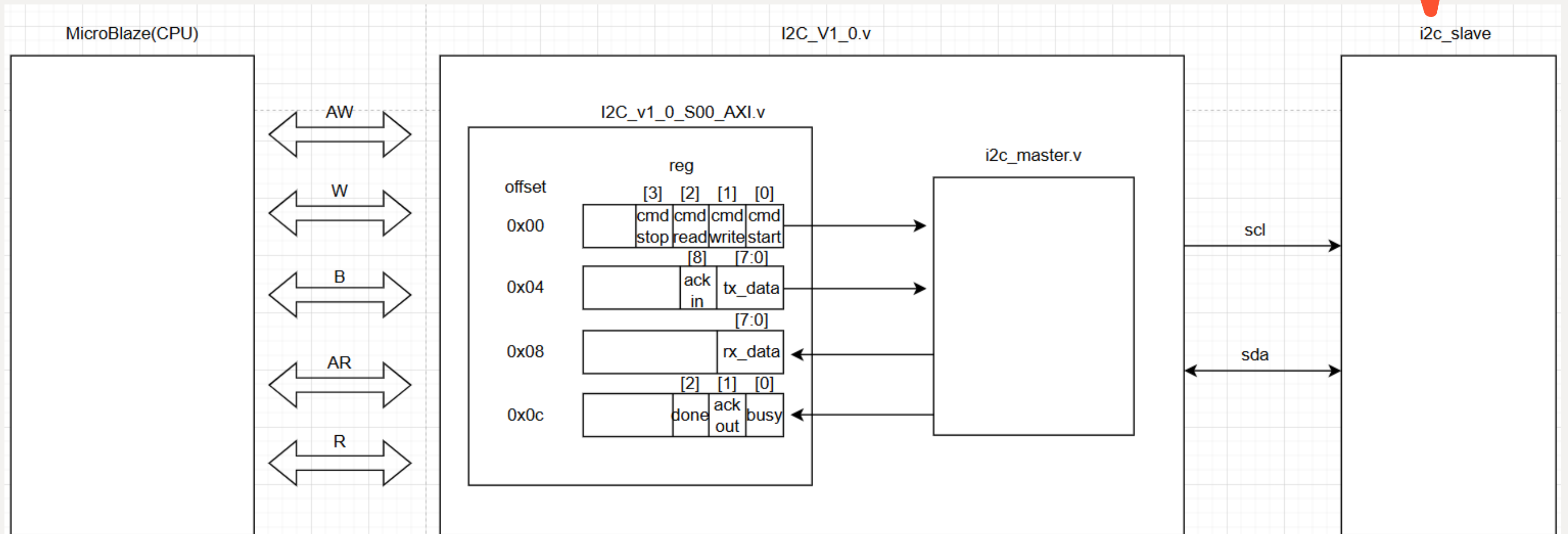
start 버튼 push
 → sw(GPIOA)에 있는 data를 전송
 (Master IP가 I2C 버스를 점유하고 해당 주소를 가진 Slave를 호출)

write 버튼 push
 → sw(GPIOA)에 있는 data를 전송(Master IP가 8비트 신호를 생성
 해 Slave의 내부 메모리에 기록)

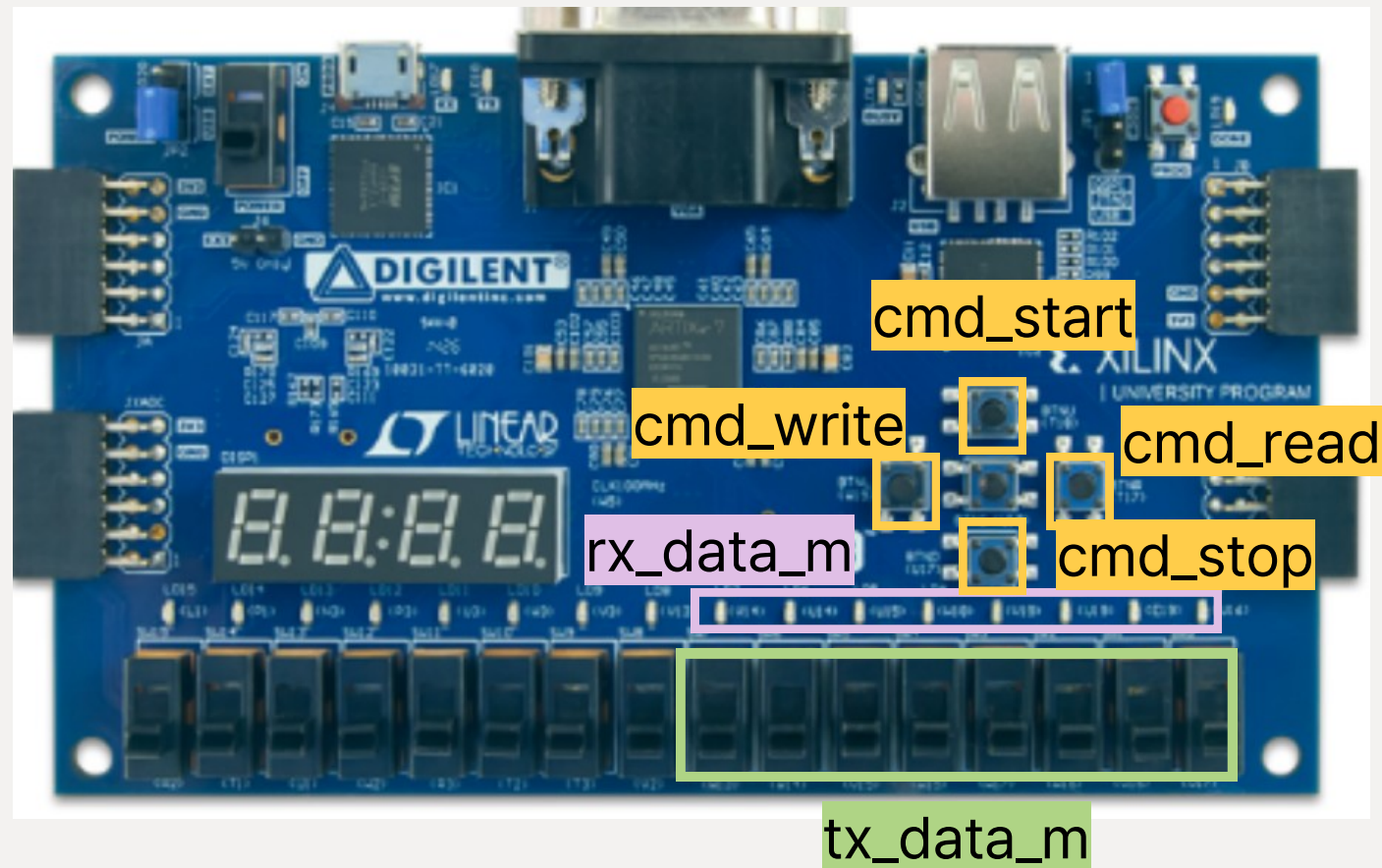
read 버튼 push
 → Slave에 저장된 데이터를 읽어와 LED(GPIOB)에 표시

stop 버튼 push
 → 통신 종료(Master IP가 SDA 라인을 High로 올려 종료)

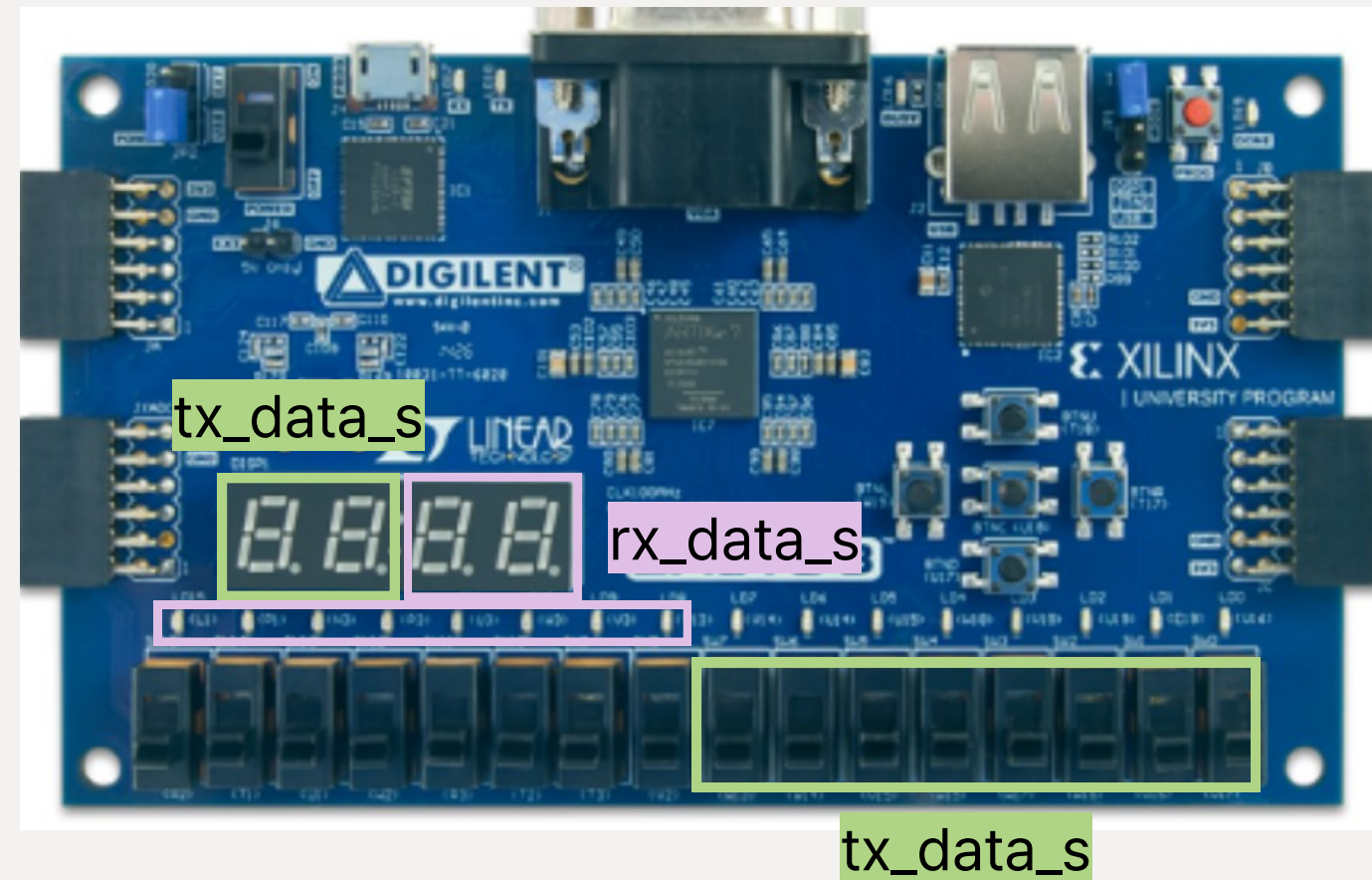
: AXI4-lite 기반 I2C Peripheral 설계



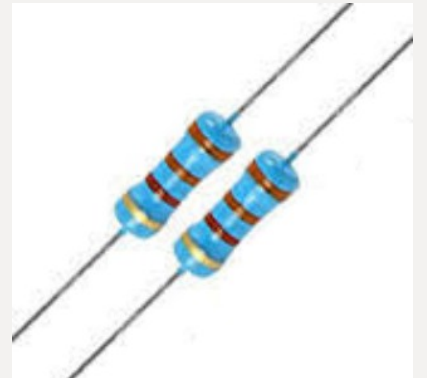
: AXI4-lite 기반 I2C Peripheral 설계



< Master >



< Slave >



< Resistor >

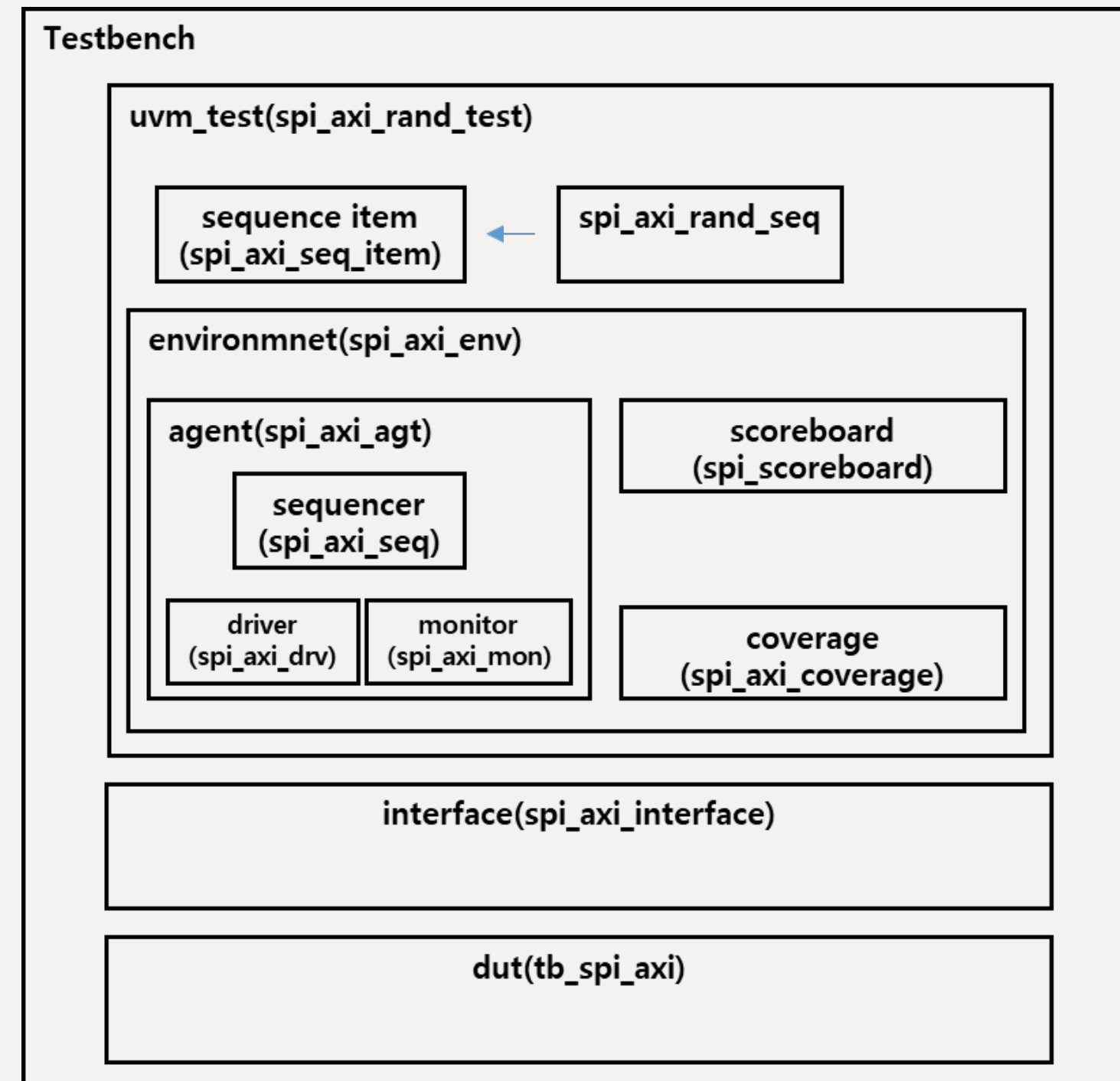
Chapter 3

Demo

: AXI4-lite 기반 I2C Peripheral 설계



UVM



```
==== AXI Scoreboard Summary ====  
Total Checked: 1000  
PASS: 1000  
FAIL: 0  
TEST PASSED: 1000 all matches detected!
```



```
==== AXI Coverage Summary ====  
Overall: 100.0%  
tx_data_m : 100.0%  
tx_data_s : 100.0%  
==== AXI Coverage Summary ====
```

```
covergroup spi_cg;  
  cp_m_tx_data: coverpoint tx.wdata[7:0] {  
    bins low_m = {[8'h00:8'h3f]};  
    bins mid_m = {[8'h40:8'hbf]};  
    bins high_m = {[8'hc0:8'hff]};  
  }  
  cp_s_tx_data: coverpoint tx.s_tx_data {  
    bins low_s = {[8'h00:8'h3f]};  
    bins mid_s = {[8'h40:8'hbf]};  
    bins high_s = {[8'hc0:8'hff]};  
  }  
endgroup
```


Trouble Shooting

AXI4_Lite - SPI, I2C

- **문제 상황** SPI와 I2C Master는 start 신호는 High → Low
AXI-Lite: CPU가 TX_START 레지스터의 8번 비트에 1을 씀.

- **원인 분석** 그러나, FPGA와 CPU의 입장에서는 High가 유지되는 그 시간이 매우 긴 시간이기에 무한 반복할 수 있다

- **문제 해결** start를 pulse 신호로 만들어주는 것이 중요
→ 소프트웨어 or 하드웨어 수정

- **느낀점** 처음에는 start pulse를 소프트웨어로 구현
→ read 동작이 원활하지는 않았음
→ hardware에서 start pulse를 구현
→ hardware가 잘 구현되어서 이전에 프로젝트를 진행했는데 이걸 수정하는 게 맞는가에 대한 의문이 생겼고, 실제로 어떤 게 정답일지 오래 고민

추가로, SoC나 계층형 아키텍처에 대해 미숙해서 베릴로그 때처럼 좋은 방향으로 코드를 짤 게 아니라 일단 어떻게든 완성하는 느낌으로 프로젝트를 진행하게 된 게 아쉬웠습니다.



발표를 들어주셔서 감사합니다